Hands-on Activity 5.2	
Basic Gameplay	
Course Code: CPE 411	**Program:** Computer Engineering
Course Title: Interactive Extended Reality	**Date Performed:** 31/03/2026
Section: 22S2	**Date Submitted:** 31/03/2026
Name(s): Ross Andrew Bulaong	**Instructor:** Engr. Jimlord M. Quejado

1. Discussion

With a controllable character in place, it's time to create **Basic Gameplay**—the rules, goals, and feedback that turn movement into a meaningful challenge. In this activity you'll design collectible items, hazards, win-lose conditions, and an on-screen score display. You'll script interactions using colliders and triggers, update UI elements in real time, and structure your code so new mechanics can be added easily. When you're done, your scene will feel like a complete mini-game that rewards skillful play.

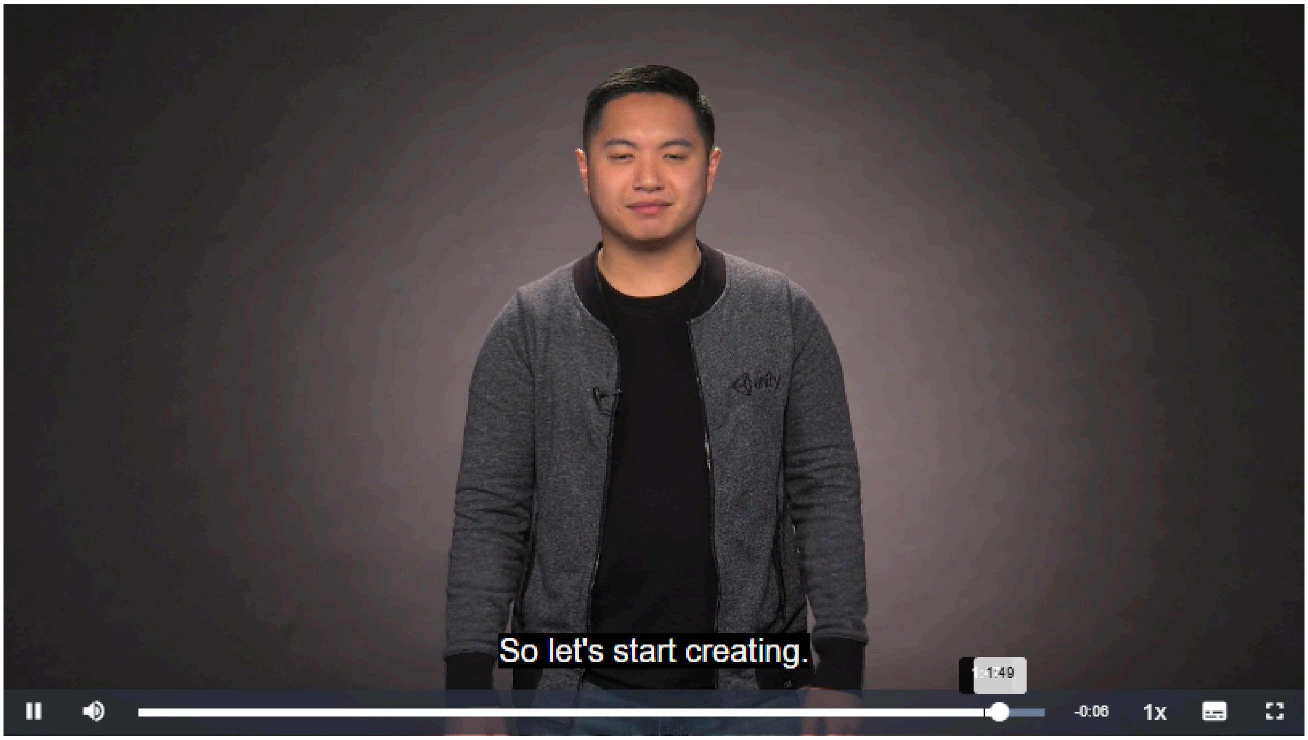
2. Materials and Equipment

Unity Hub and Unity Editor
Learn Unity Website (Unity Essentials Pathway - Learn Game Development for Beginners | Unity Learn)
Computer with Network Connection

3. Procedure

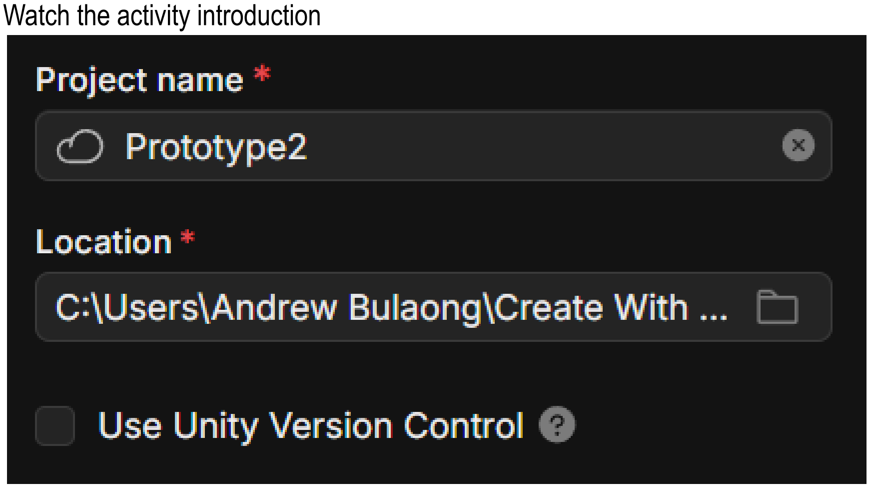
1. Unit 2 - Introduction

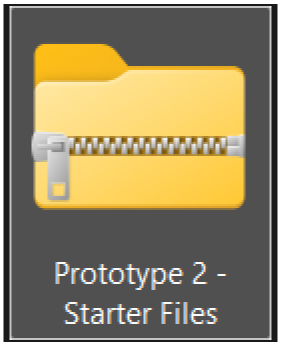
Q&A (0)

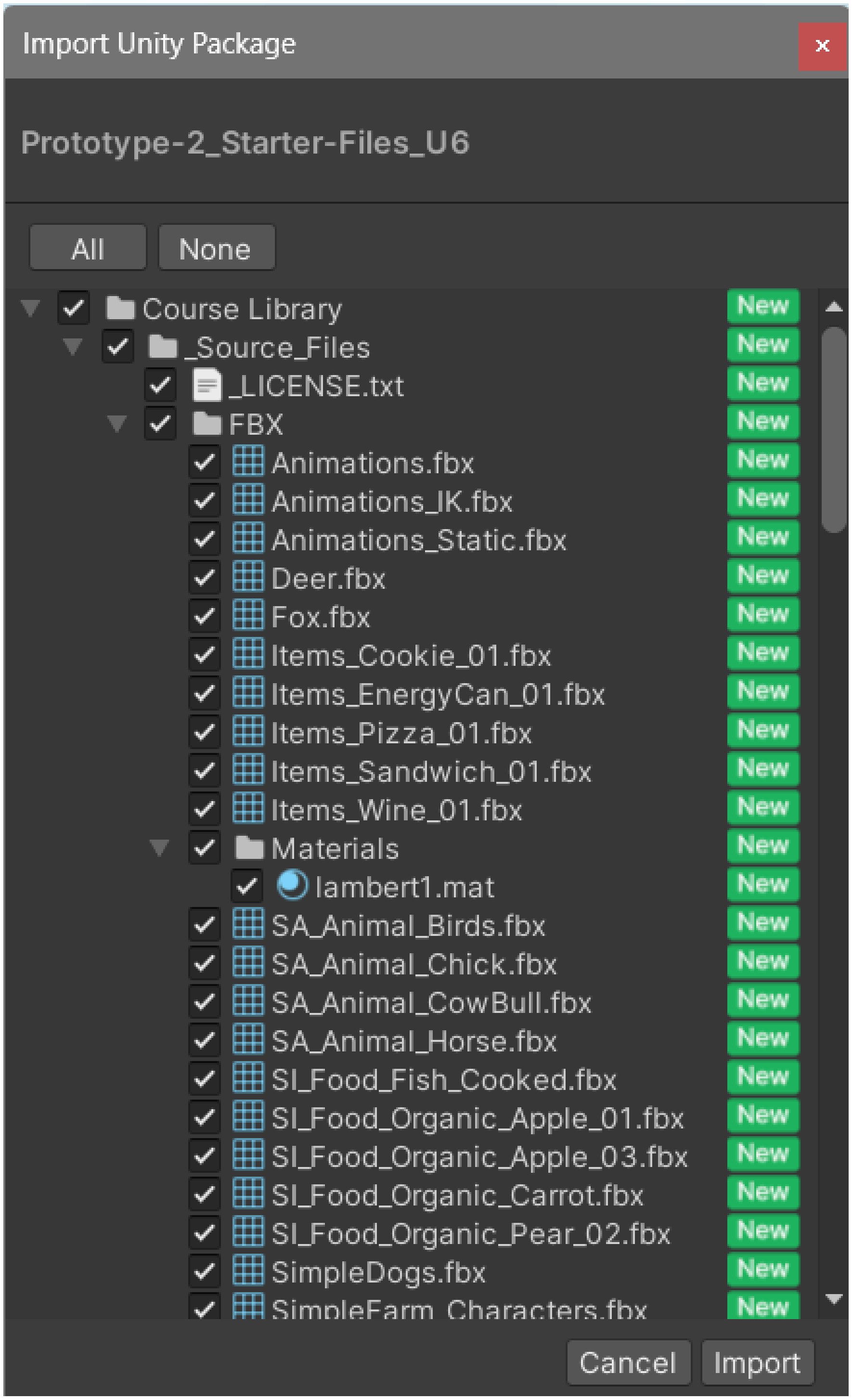


Mark Step As Complete

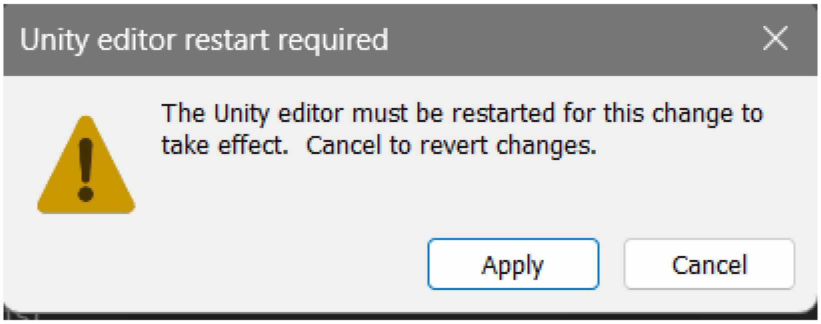
Watch the activity introduction



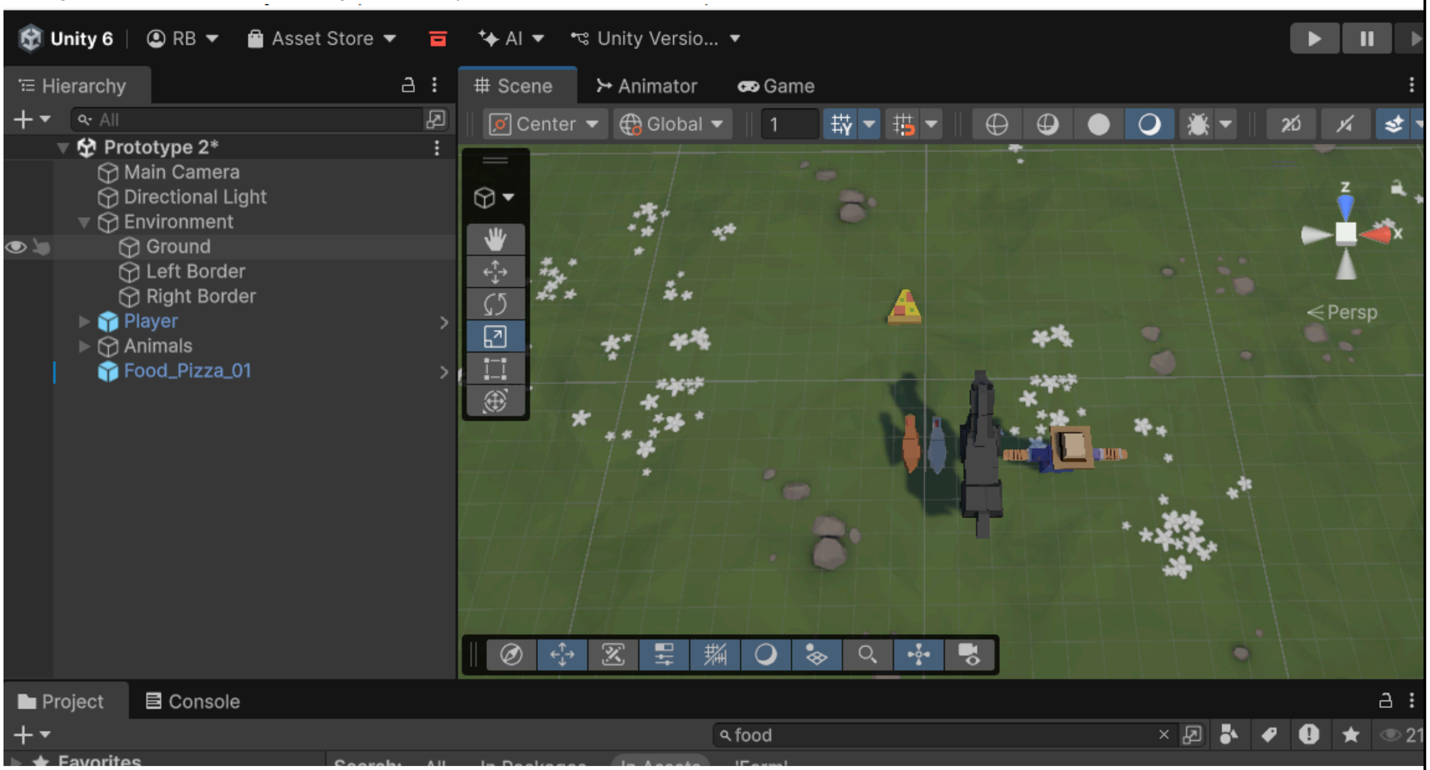


Create project and download the assets as instructed


Import the files



Change the active input handling of the player to “Both”



Add player, animals, and food, which are scaled to be easel seen from above

```
using UnityEngine;

public class NewMonoBehaviourScript : MonoBehaviour
{
    void Start()
    {

    }

    public float horizontalInput;

    void Update()
    {


        horizontalInput = Input.GetAxis("Horizontal");
    }
}

```

Horizonta L Input 0.7649645

Create a script that takes the player's horizontal input (left and right) and attach the controller to the player character

```
1   using UnityEngine;
2
3   public class NewMonoBehaviourScript : MonoBehaviour
4   {
5       void Start()
6       {
7
8       }
9
10      public float horizontalInput;
11      public float speed;
12
13      void Update()
14      {
15          horizontalInput = Input.GetAxis("Horizontal");
16          transform.Translate(Vector3.right * horizontalInput * speed * Time.deltaTime)
17      }
18  }
19
```

Move the character left and right based on the speed and horizontal input values (I changed the typo after this screenshot, and added a semi colon on the end of the 16th line)

```
using UnityEngine;

public class NewMonoBehaviourScript : MonoBehaviour
{
    void Start()
    {

    }

    public float horizontalInput;
    public float speed;

    void Update()
    {
        horizontalInput = Input.GetAxis("Horizontal");
        transform.Translate(Vector3.right * horizontalInput * speed * Time.deltaTime);

        if (transform.position.x < -10){
            transform.position = new Vector3(-10, transform.position.y,transform.position.z);
        }
    }
}
```

Create an automatic boundary script that returns the player back to the original x position -10

```
using UnityEngine;

public class PlayerController : MonoBehaviour
{
    void Start()
    {

    }

    private float horizontalInput;
    public float speed = 10f;
    public float xRange = 10f;

    void Update()
    {
        horizontalInput = Input.GetAxis("Horizontal");
        transform.Translate(Vector3.right * horizontalInput * speed * Time.deltaTime);

        if (transform.position.x < -xRange){
            transform.position = new Vector3(-xRange, transform.position.y,transform.position.z);
        }
        if (transform.position.x > xRange){
            transform.position = new Vector3(xRange, transform.position.y, transform.position.z);
        }
    }
}
```

Create an xRange variable for easy Inspector-changing, and add new logic for when the player's position is greater than the possible range

```
using UnityEngine;

public class MoveForward : MonoBehaviour
{

    void Start()
    {

    }

    public float speed = 40.0f;

    void Update()
    {
        transform.Translate(Vector3.forward * Time.deltaTime * speed);
    }
}
```

Create a script that moves the object its attached to forward every frame

```
using UnityEngine;

public class PlayerController : MonoBehaviour
{
    void Start()
    {

    }

    private float horizontalInput;
    public float speed = 10f;
    public float xRange = 10f;
    public GameObject projectilePrefab ;

    void Update()
    {
        horizontalInput = Input.GetAxis("Horizontal");
        transform.Translate(Vector3.right * horizontalInput * speed * Time.deltaTime);

        if (transform.position.x < -xRange){
            transform.position = new Vector3(-xRange, transform.position.y,transform.position.z);
        }
        if (transform.position.x > xRange){
            transform.position = new Vector3(xRange, transform.position.y, transform.position.z);
        }
    }
}
```

Create a public variable to assign the created prefab to the player controller

```
using UnityEngine;

public class PlayerController : MonoBehaviour
{
    void Start()
    {

    }

    private float horizontalInput;
    public float speed = 10f;
    public float xRange = 10f;
    public GameObject projectilePrefab ;

    void Update()
    {
        horizontalInput = Input.GetAxis("Horizontal");
        transform.Translate(Vector3.right * horizontalInput * speed * Time.deltaTime);

        if (transform.position.x < -xRange){
            transform.position = new Vector3(-xRange, transform.position.y,transform.position.z);
        }
        if (transform.position.x > xRange){
            transform.position = new Vector3(xRange, transform.position.y, transform.position.z);
        }
        if(Input.GetKeyDown(KeyCode.Space)){
            // Launch a projectile from the player (farmer)
        }
    }
}
```

```
using UnityEngine;

public class PlayerController : MonoBehaviour
{
    void Start()
    {

    }

    private float horizontalInput;
    public float speed = 10f;
    public float xRange = 10f;
    public GameObject projectilePrefab ;

    void Update()
    {
        horizontalInput = Input.GetAxis("Horizontal");
        transform.Translate(Vector3.right * horizontalInput * speed * Time.deltaTime);

        if (transform.position.x < -xRange){
            transform.position = new Vector3(-xRange, transform.position.y,transform.position.z);
        }
        if (transform.position.x > xRange){
            transform.position = new Vector3(xRange, transform.position.y, transform.position.z);
        }
        if(Input.GetKeyDown(KeyCode.Space)){
            Instantiate(projectilePrefab, transform.position, projectilePrefab.transform.rotation);
        }
    }
}
```

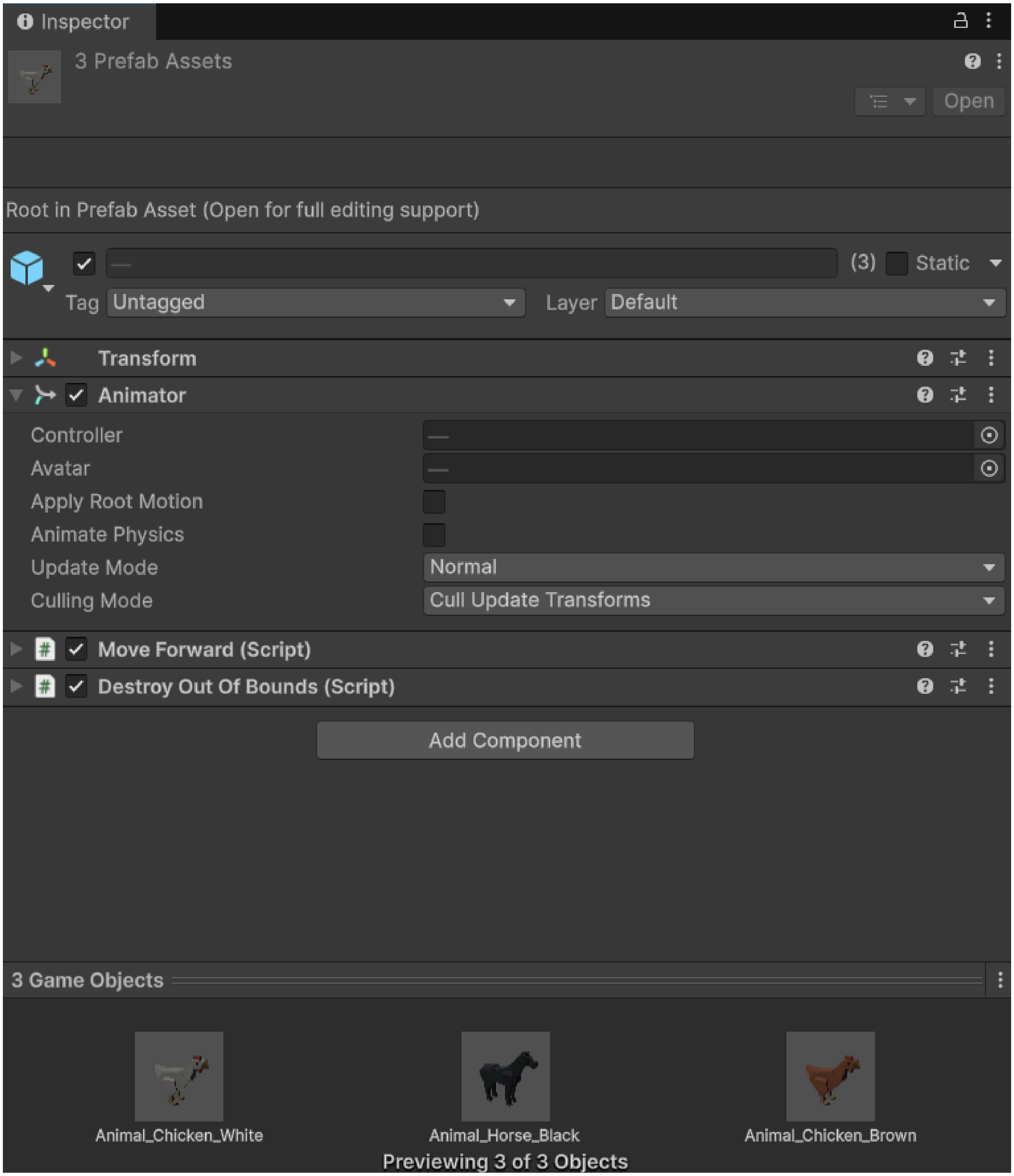
Create logic for launching projectiles on the player controller script

Rotate all animals 180 degrees, add the move forward script to each one and edit the speed to match, then make them prefabs

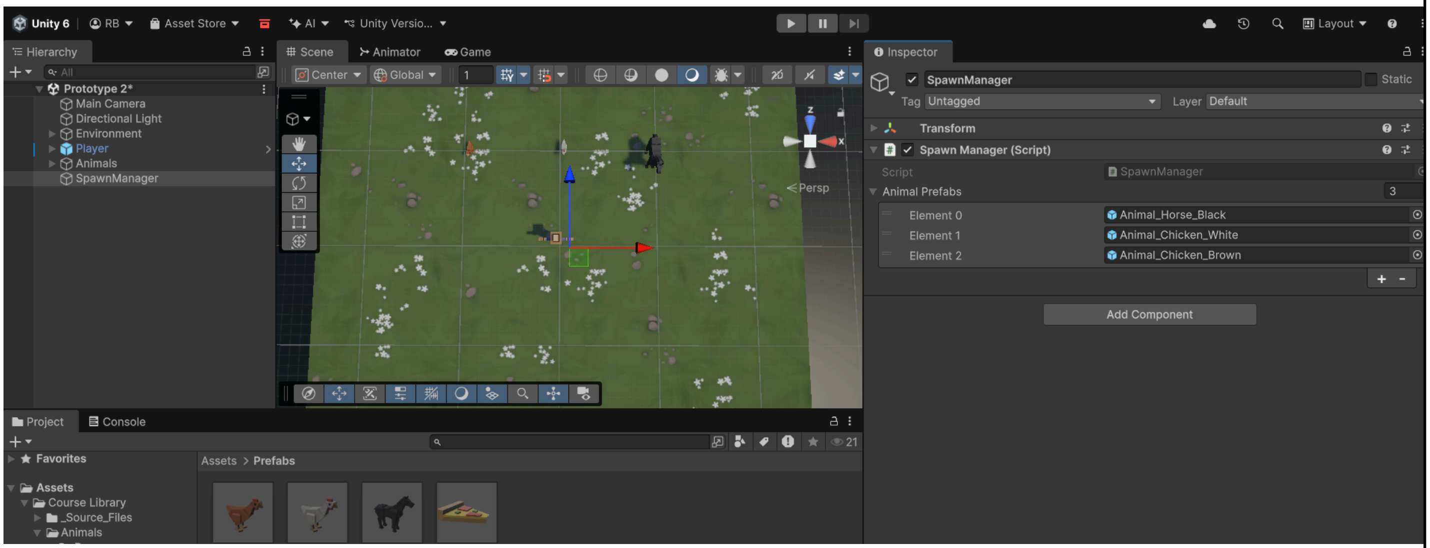
```csharp
using UnityEngine;

public class DestroyOutOfBounds : MonoBehaviour
{

    void Start()
    {

    }

    private float topBound = 30;

    void Update()
    {
        if (transform.position.z > topBound){
            Destroy(gameObject);
        }
    }
}
```

Create a script that destroys all objects off screen (out of bounds) so it doesn't lag the game

```csharp
using UnityEngine;

public class DestroyOutOfBounds : MonoBehaviour
{

    void Start()
    {

    }

    private float topBound = 30;
    private float lowerBound = -10;

    void Update()
    {
        if (transform.position.z > topBound){
            Destroy(gameObject);
        }
        else if (transform.position.z < lowerBound){
            Destroy(gameObject);
        }
    }
}
```



Create logic for the animals (which this script is also attached to) to be destroyed after they are off screen (lowerBound)



SpawnManager

```
using UnityEngine;

public class SpawnManager : MonoBehaviour
{
    void Start()
    {

    }

    public GameObject[] animalPrefabs;

    void Update()
    {

    }
}

```

Create SpawnManager gameObject and script; create a public array where the amount of animals can be assigned (as well as the animals themselves)

```csharp
using UnityEngine;

public class SpawnManager : MonoBehaviour
{
    void Start()
    {

    }

    public GameObject[] animalPrefabs;
    public int animalIndex;

    void Update()
    {
        if (Input.GetKeyDown(KeyCode.S)){
            Instantiate(animalPrefabs[animalIndex], new Vector3(0,0,20),
            animalPrefabs[animalIndex].transform.rotation);
        }
    }
}
```

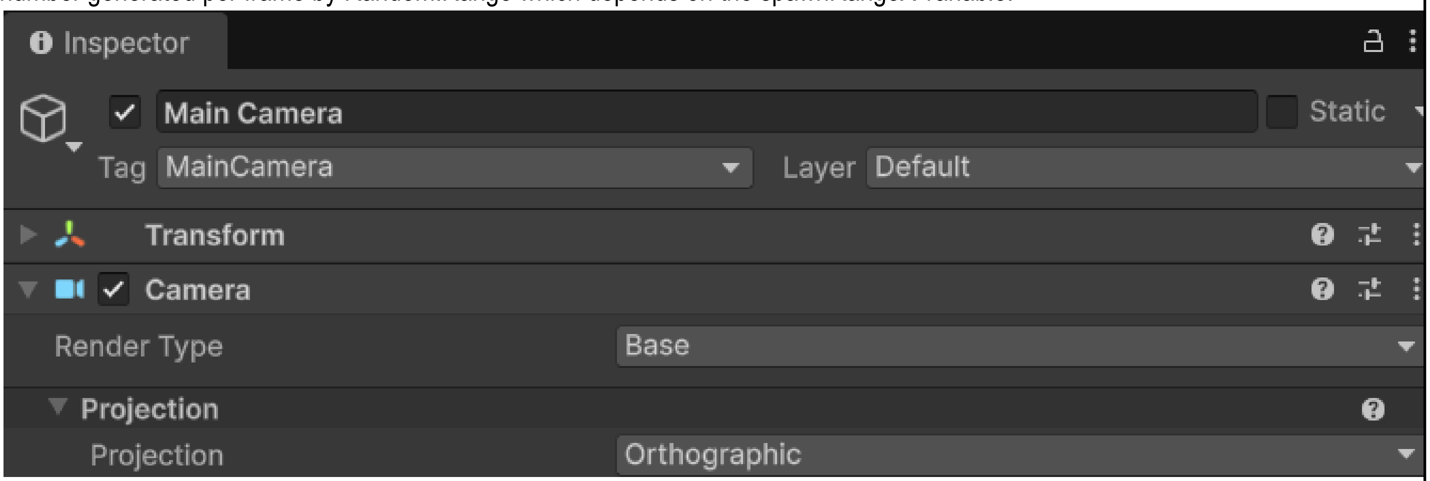
Create a public variable animalIndex to control which animal spawns when S is pressed (based on the index, starting from 0)

```csharp
using UnityEngine;

public class SpawnManager : MonoBehaviour
{
    void Start()
    {

    }

    public GameObject[] animalPrefabs;


    void Update()
    {
        int animalIndex = Random.Range(0,animalPrefabs.length)
        if (Input.GetKeyDown(KeyCode.S)){
            Instantiate(animalPrefabs[animalIndex], new Vector3(0,0,20),
            animalPrefabs[animalIndex].transform.rotation);
        }
    }
}
```

Remove the manual assignment of animalIndex, instead, create a randomized index by creating a random value for animalIndex per frame based on the length of the array animalPrefabs. (i changed length to Length as it threw an error due to miscapitalization)

```
using UnityEngine;

public class SpawnManager : MonoBehaviour
{
    void Start()
    {

    }

    public GameObject[] animalPrefabs;
    private float spawnRangeX = 20;
    private float spawnPosZ = 20;


    void Update()
    {
        int animalIndex = Random.Range(0,animalPrefabs.Length);
        Vector3 spawnPos = new Vector3(Random.Range(-spawnRangeX, spawnRangeX),0,spawnPosZ);
        if (Input.GetKeyDown(KeyCode.S)){
            Instantiate(animalPrefabs[animalIndex], spawnPos,
            animalPrefabs[animalIndex].transform.rotation);
        }
    }
}

```

Create a script that randomizes the spawn location of each animal after pressing S based on the private floats spawnRangeX and spawnPosZ; where all animals spawn from the upper bound of the game, and the range where they spawn depends on the random number generated per frame by Random.Range which depends on the spawnRangeX variable.



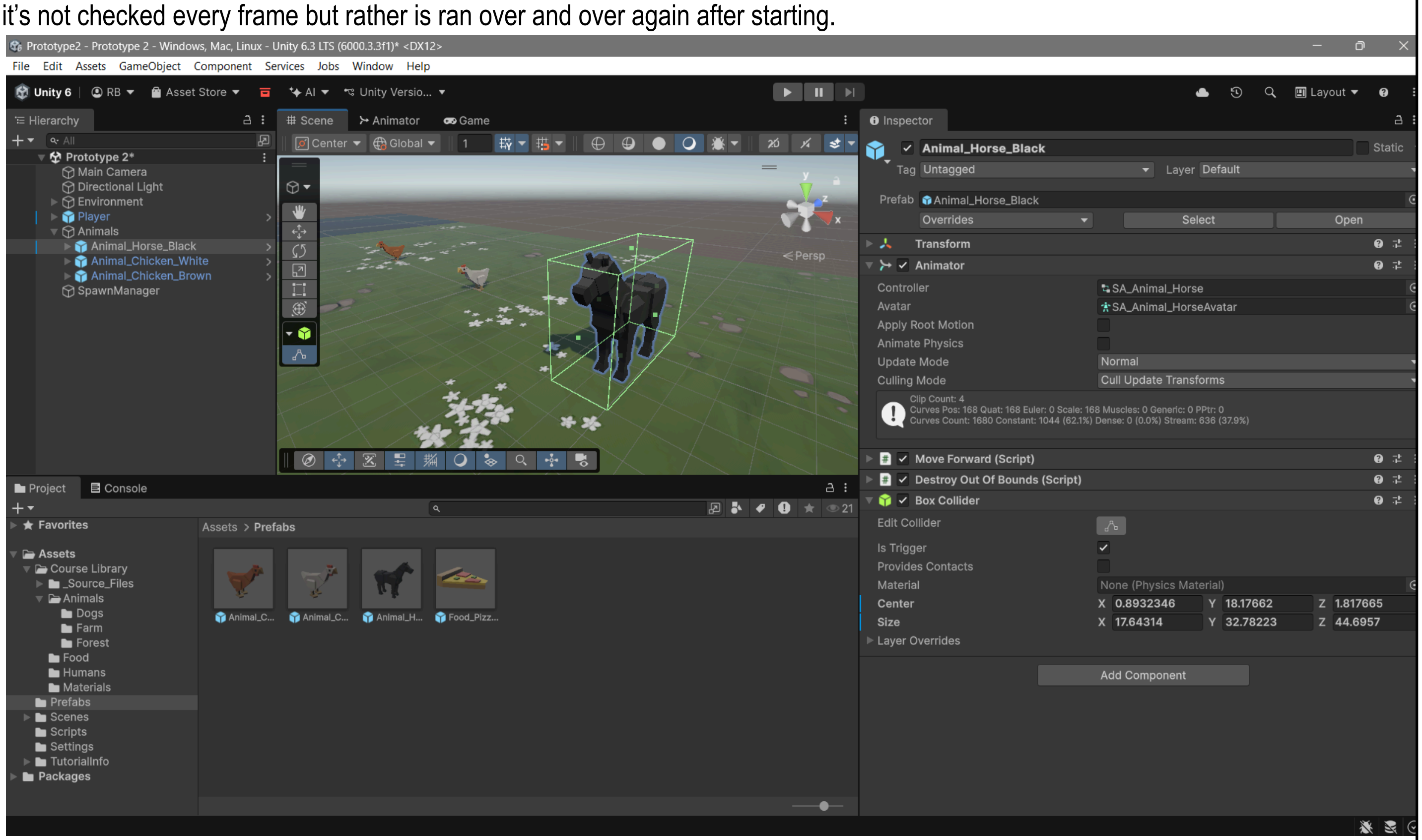
Change the projection of the camera to Orthographic as instructed.

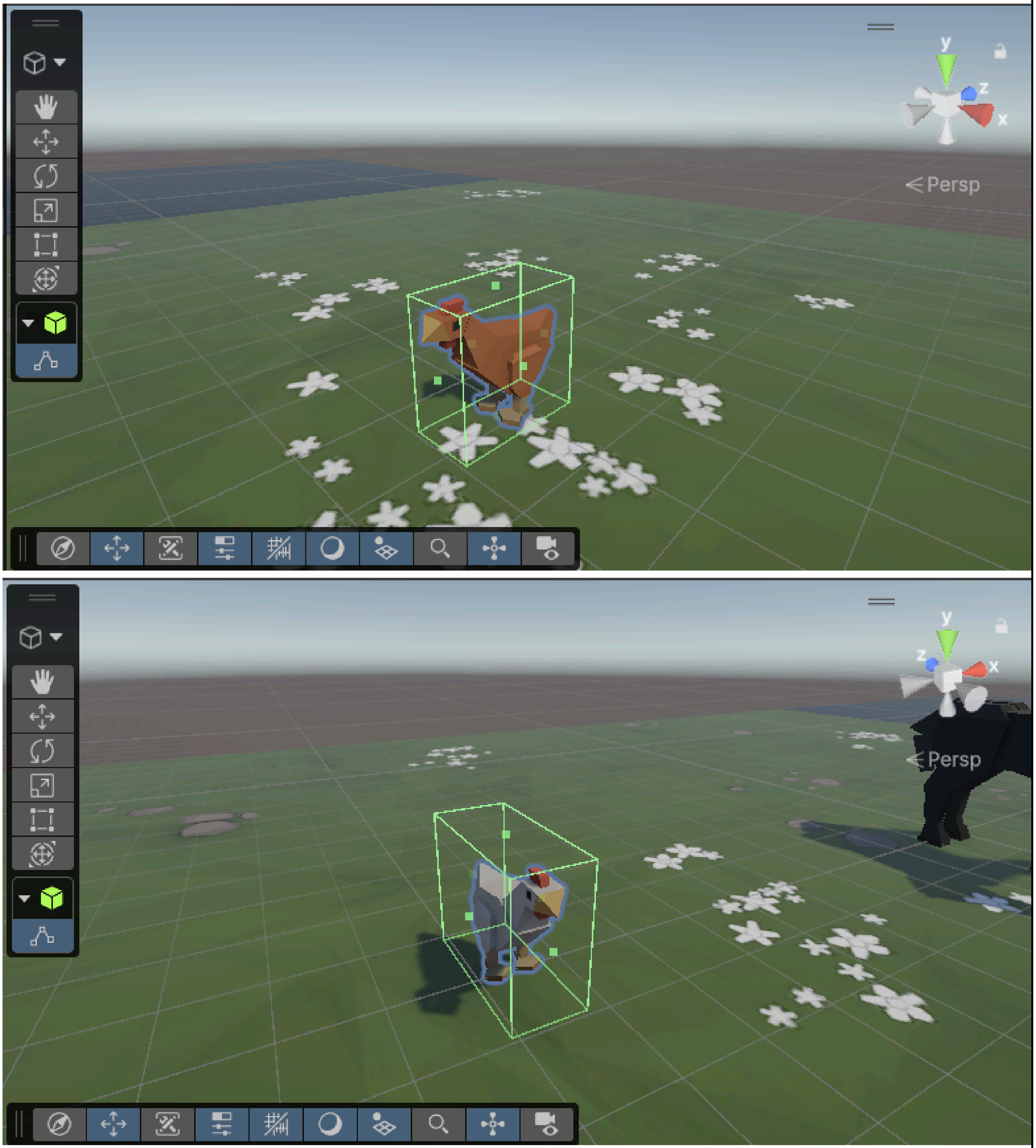
```
using UnityEngine;

public class SpawnManager : MonoBehaviour
{
    void Start()
    {

    }

    public GameObject[] animalPrefabs;
    private float spawnRangeX = 20;
    private float spawnPosZ = 20;


    void Update()
    {
        if (Input.GetKeyDown(KeyCode.S)){
           SpawnAnimal();
        }
    }

    void SpawnRandomAnimal(){
        int animalIndex = Random.Range(0,animalPrefabs.Length);
        Vector3 spawnPos = new Vector3(Random.Range(-spawnRangeX, spawnRangeX),0,spawnPosZ);
        Instantiate(animalPrefabs[animalIndex], spawnPos,
        animalPrefabs[animalIndex].transform.rotation);
    }
}
```

Create a function for animal spawns to clean and modularize the code

```csharp
using UnityEngine;

public class SpawnManager : MonoBehaviour
{
    void Start()
    {
        InvokeRepeating("SpawnRandomAnimal",startDelay,spawnInterval);
    }

    public GameObject[] animalPrefabs;
    private float spawnRangeX = 20;
    private float spawnPosZ = 20;
    private float startDelay = 2;
    private float spawnInterval = 1.5f;


    void Update()
    {
    }

    void SpawnRandomAnimal(){
        int animalIndex = Random.Range(0,animalPrefabs.Length);
        Vector3 spawnPos = new Vector3(Random.Range(-spawnRangeX, spawnRangeX),0,spawnPosZ);
        Instantiate(animalPrefabs[animalIndex], spawnPos,
        animalPrefabs[animalIndex].transform.rotation);
    }
}

```

Change the logic so that the automatic spawning of animals happens at fixed intervals that start as soon as the game loads; meaning it's not checked every frame but rather is ran over and over again after starting.





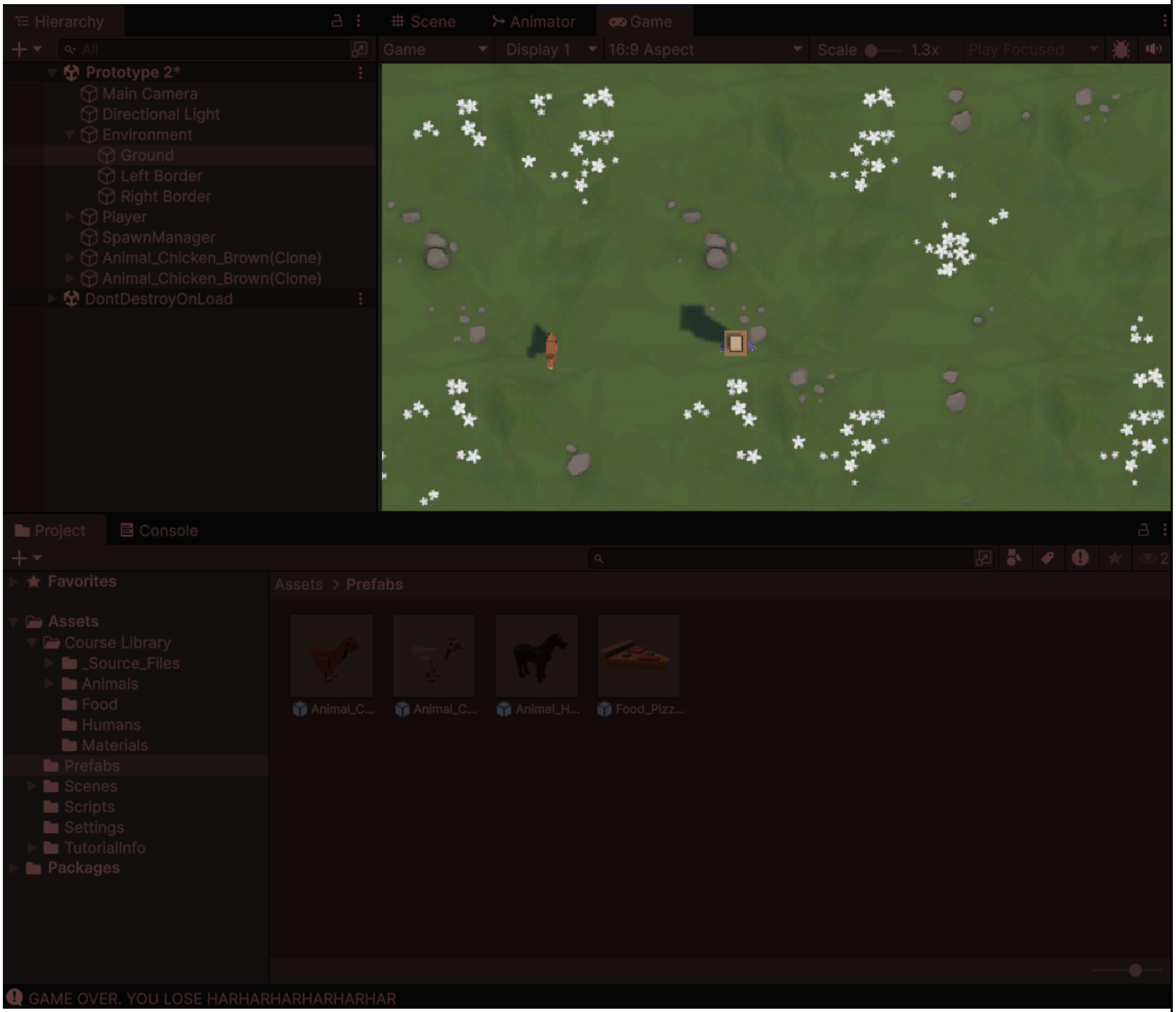
Create colliders for all the animals / projectile and tick the IsTrigger checkbox

```
using UnityEngine;

public class DetectCollision : MonoBehaviour
{

    void Start()
    {

    }

    void Update()
    {

    }

    void OnTriggerEnter(Collider other){
        Destroy(gameObject);
        Destroy(other.gameObject)
    }
}

```

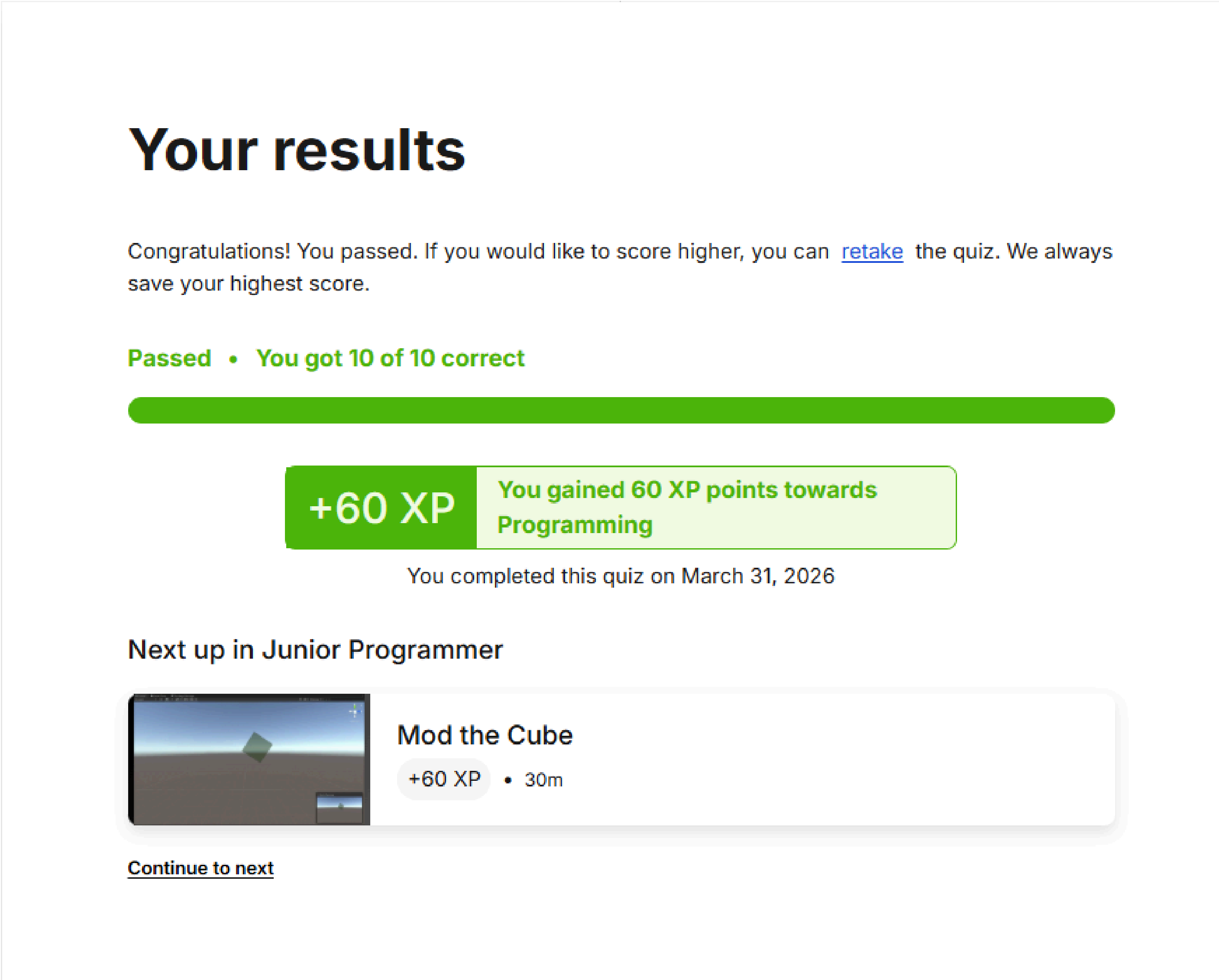
Create a script that destroys colliding objects (animals and projectiles), then attach the script to each animal

```
using UnityEngine;

public class DestroyOutOfBounds : MonoBehaviour
{

    void Start()
    {

    }

    private float topBound = 30;
    private float lowerBound = -10;

    void Update()
    {
        if (transform.position.z > topBound){
            Destroy(gameObject);
        }
        else if (transform.position.z < lowerBound){
            Destroy(gameObject);
            Debug.Log("GAME OVER. YOU LOSE HARHARHARHARHARHAR");
        }
    }
}

```

Add a game over message if an animal reaches the out of bounds section.



Game test: SUCCESS!



Quiz results

4. Conclusion

I loved this laboratory and I learned a heap ton of new techniques, ideas, as well as coding knowledge. I personally never knew that InvokeRepeating was a function, which was the highlight of my learnings. This game taught me how to handle a “Space Invader” type of game/gameplay, which I'll use on my future games that I will develop.

5. Assessment Rubric

Lab Activity Rubric

Criteria	Ratings						Pts
SO 7 PI 1 **Student Outcome 7.1** Acquire and apply new knowledge from outside sources. threshold: 4.8 pts	6 pts Excellent | Educational interests and pursuits exist and flourish outside classroom requirements,knowledge and/or experiences are pursued independently and applies knowledge learned into practice	5 pts Good | Educational interests and pursuits exist and flourish outside classroom requirements,knowledge and/or experiences are pursued independently	4 pts Satisfactory | Look beyond classroom requirements, showing interest in pursuing knowledge independently	3 pts Unsatisfactory | Begins to look beyond classroom requirements, showing interest in pursuing knowledge independently	2 pts Poor | Relies on classroom instruction only	1 pts Very Poor | No initiative or interest in acquiring new knowledge	6 pts
SO 7 PI 2 **Student Outcome 7.2** Learn independently threshold: 4.8 pts	6 pts Excellent | Completes an assigned task independently and practices continuous improvement	5 pts Good | Completes an assigned task without supervision or guidance	4 pts Satisfactory | Requires minimal guidance to complete an assigned task	3 pts Unsatisfactory | Requires detailed or step-by-step instructions to complete a task	2 pts Poor | Shows little interest to complete a task independently	1 pts Very Poor | No interest to complete a task independently	6 pts
SO 7 PI 3 **Student Outcome 7.3** Critical thinking in the broadest context of technological change threshold: 4.8 pts	6 pts Excellent | Synthesizes and integrates information from a variety of sources; formulates a clear and precise perspective; draws appropriate conclusions	5 pts Good | Evaluate information from a variety of sources; formulates a clear and precise perspective.	4 pts Satisfactory | Analyze information from a variety of sources; formulates a clear and precise perspective.	3 pts Unsatisfactory | Apply the gathered information to formulate the problem	2 pts Poor | Gather and summarized the information from a variety of sources but failed to formulate the problem	1 pts Very Poor | Gather information from a variety of sources	6 pts
SO 7 PI 4 **Student Outcome 7.4** Creativity and adaptability to new and emerging technologies threshold: 4.8 pts	6 pts Excellent | Ideas are combined in original and creative ways in line with the new and emerging technology trends to solve a problem or address an issue.	5 pts Good | Ideas are creative and adapt the new knowledge to solve a problem or address an issue	4 pts Satisfactory | Ideas are creative in solving a problem, or address an issue	3 pts Unsatisfactory | Shows some creative ways to solve the problem	2 pts Poor | Shows initiative and attempt to develop creative ideas to solve the problem	1 pts Very Poor | Ideas are copied or restated from the sources consulted	6 pts

Total Points: 24